Lecture 30: Diffusion IV

Text to image: conditioning and guidance

Some panels are stills from Welch Labs / 3Blue1Brown (2025).

Everything so far has been unconditional

We can start from noise and land on the data. The digits in L29 were digits - but we could not ask for a **seven**.

Every model in this chapter has drawn *a* sample from $p(\mathbf{x})$. What people actually want is a sample from

$$p(\mathbf{x} \mid \text{"a lone tree in the desert"})$$

Two problems, and they are separate:

1. How does a **sentence** become something a convolutional network can use?
2. Once it is in there, how do we make the model actually **obey** it?

The second turns out to be the harder one, and the answer is surprisingly blunt.

Outline

One space for words and pictures

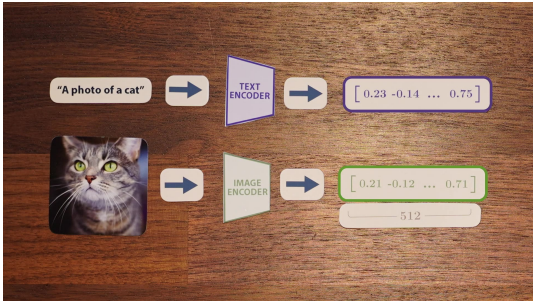
Conditioning

Classifier-free guidance

One space for words and pictures

Before we can steer with text, text and images must live somewhere in common.

CLIP: two encoders, one destination



Welch Labs / 3Blue1Brown (2025)

CLIP - Contrastive Language-Image Pre-training (Radford et al., 2021) - is two separate networks:

- a **text** encoder: caption → vector
- an **image** encoder: picture → vector

Both output vectors of the same length (512 in the original), into the same space.

The goal: an image and *its own caption* should land in nearly the same place. Trained on 400 million image-caption pairs scraped from the web.

Contrastive learning, in one idea

We have no labels saying "this vector should be at coordinates $(0.3, -1.1, \dots)$". All we have is which caption belongs to which image. That is enough.

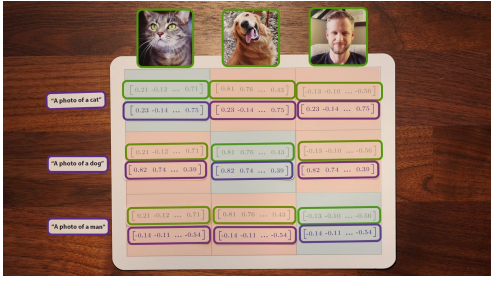
Pull together what belongs together. **Push apart** everything else.

Nobody says where anything goes. The geometry is a *consequence* of thousands of these pulls and pushes settling against each other.

Where the negatives come from is the clever part. Take a batch of N pairs. Each image has one correct caption and $N-1$ wrong ones - already in the batch, free.

Bigger batches mean harder negatives, which is one reason CLIP needed to be trained at scale.

The training objective is a matrix



Welch Labs / 3Blue1Brown (2025)

Images as columns, captions as rows. Every cell is the similarity of one image with one caption.

- the **diagonal** holds the true pairs → maximize
- everything **off-diagonal** is a mismatch → minimize

That is the whole loss: a classification problem where the "classes" are "which of these N captions is mine?", run in both directions at once.

The **C** in CLIP is this contrast.

The similarity is one you already know

CLIP scores a pair with **cosine similarity** - the cosine of the angle between the two vectors:

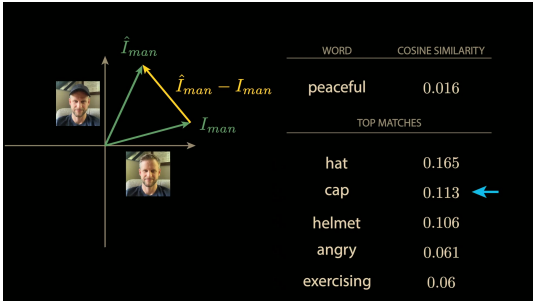
$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \cos \vartheta$$

This is the dot product from L24, normalized. There we used it to ask how much one token should attend to another; here we ask how much a caption matches a picture. Same operation, same meaning: *alignment*.

Dividing out the magnitudes means only **direction** carries meaning. Two vectors pointing the same way are "the same idea", regardless of length.

You met cosine once before as a distance option in clustering (L13). This is where it starts earning its keep.

The space learned arithmetic nobody asked for



Welch Labs / 3Blue1Brown (2025)

Encode a photo of a person **wearing a hat**. Encode the same person **without**. Subtract the two image vectors.

Now search the *text* side for the caption closest to that difference.

The top match is "**hat**" (0.165), then "cap", then "helmet".

Nothing in the training objective asked for this. Pull matches together, push mismatches apart, and a space falls out where **directions correspond to concepts** - and where you can cross from images to words and back.

Two consequences

Classification for free. Encode an image. Encode "a photo of a {cat, dog, truck, ...}". Pick the caption with the highest similarity.

No training on those classes - **zero-shot**. CLIP was competitive with supervised models on datasets it had never been fitted to.

But it only goes one way. Both encoders map *into* the space. Neither comes back out.

CLIP can tell you an image matches "a lone tree in the desert". It cannot draw one.

We have a model that turns pictures into meaning, and a model that turns noise into pictures.

Bolt them together.

Conditioning

Getting the prompt inside the network. The easy half.

Give the denoiser one more input

The network so far is $\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$. Add the prompt as a third argument:

$$\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t,\ t,\ \mathbf{c})$$

Train exactly as before, on image-caption pairs, with the same $\mathcal{L}_{\text{simple}}$ from L28.

The bet: knowing the image is *of a tree* should help the model predict which noise was added to it. Better denoising, and a model that has learned to use text.

We have done this before. Conditioning on t in L27 was the same move: an extra input that tells the network which situation it is in. Text is just a much richer t.

What exactly goes in?

The obvious answer - "the CLIP text vector" - is **not** what production models do.

A single pooled 512-d vector squeezes the whole sentence into one point. "A dog chasing a cat" and "a cat chasing a dog" become nearly identical.

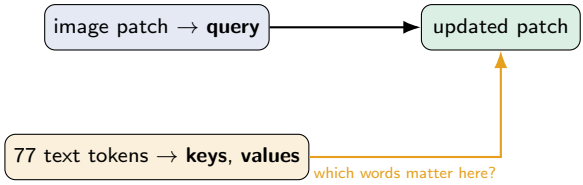
Stable Diffusion takes the layer *before* the pooling: a **sequence** of per-token vectors, $\mathbf{77} \times \mathbf{768}$. One vector per token, not one per sentence.

Why 768 and not the 512 from two slides ago: Stable Diffusion uses a larger CLIP variant (ViT-L/14, width 768) than the one in the original zero-shot demos (ViT-B/32, width 512). The 77 is CLIP's fixed maximum token count - prompts are padded or truncated to it.

This is not a detail. The mechanism on the next slide **needs** a sequence to work at all - there is nothing to look across if the prompt is a single point.

Cross-attention: how the pixels read the prompt

In L24 attention let each token look at every other token, by matching a **query** against **keys** and taking a weighted sum of **values**. Same machinery here - but the queries and the keys come from different places.



Every patch of the image asks: *which words in this prompt are about me?* A patch that is becoming sky attends to "sky"; a patch becoming bark attends to "tree".

Self-attention (L24): queries, keys and values all from one sequence.

Cross-attention: queries from the image, keys and values from the text. That is the only difference.

These layers sit inside the UNet blocks from L28. Simpler alternatives exist - concatenating the text vector, or adding it like the time embedding - and many models use several routes at once.

Doing it properly: DALL·E 2

OpenAI trained a diffusion model to **invert the CLIP image encoder**: given a CLIP embedding, produce an image that would encode to it.

CLIP maps pictures to meaning; the diffusion model maps meaning back to pictures. They called it **unCLIP** (Ramesh et al., 2022); the product was **DALL·E 2**.

Prompt adherence jumped far beyond anything before it. This is the moment text-to-image stopped being a demo.

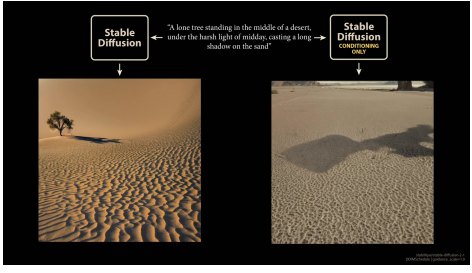
Stable Diffusion (Rombach et al., 2022) arrived around the same time from Heidelberg, worked differently in an important way - the subject of L31 - and was **open source**, which is why it is the model everyone experiments with.

Is that enough?

Predict first: we have a well-trained conditional model. Prompt: *"a lone tree standing in the middle of a desert, under the harsh light of midday, casting a long shadow on the sand."* What comes out?

Is that enough?

Predict first: we have a well-trained conditional model. Prompt: *"a lone tree standing in the middle of a desert, under the harsh light of midday, casting a long shadow on the sand."* What comes out?



Welch Labs / 3Blue1Brown (2025)

A desert. Harsh midday light. A long shadow on the sand.

No tree.

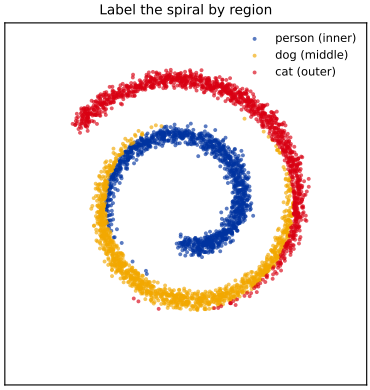
The model used the prompt - the scene is right. But it is behaving like a model that was asked for a *plausible image, given* the text, and plausible desert photographs often have no tree in them.

Conditioning tells the model what the prompt *is*. It does not make the model **care**.

Classifier-free guidance

One idea, three lines of code, and the reason your prompts work.

Back to the spiral, with labels



Split the spiral into three regions and call them classes - our stand-in for a prompt. Train exactly as in L28, with the class as an extra input.

The model now has two jobs at once: **land on the spiral** (all classes) and **land on the right part of it** (this class).

The first job is easy and dominates the loss - most of the error is in getting near the data at all. The second is a small correction on top, and it gets drowned out. That is the missing tree, in miniature.

The trick: train one model to be two models

During training, **throw the label away** on a fraction of examples (around 10-20%) and replace it with a "no class" token.

One set of weights, two behaviours:

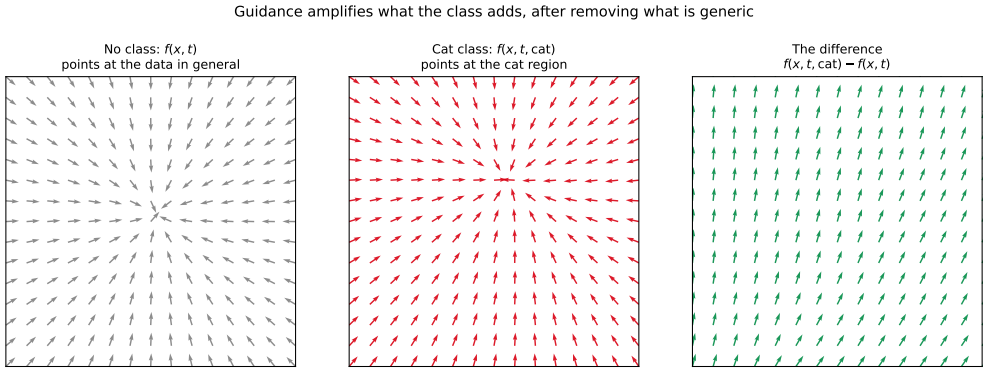
- $\varepsilon_\theta(\mathbf{x}_t, t, \mathbf{c})$ - "draw a cat"
- $\varepsilon_\theta(\mathbf{x}_t, t)$ - "draw anything"

You could train two separate networks. Dropping the label is free and gives the same thing, which is why it is universal.

Ho & Salimans (2022) - "classifier-free" because earlier work steered diffusion with a separately trained classifier, and this needs none.

Now we have both a general direction and a class-specific one, and we can compare them. **The difference between them is the part of the prediction that is about the prompt.**

Isolate the prompt's contribution, then exaggerate it



Left: without a class, the field points at the spiral in general. Middle: told "cat", it leans outward. Right: subtract them and what remains is **only** the cat-ness - the generic "get to the data" part has cancelled out.

The guidance formula

Scale that difference by α and add it back:

$$\tilde{\varepsilon} \;=\; \varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \;+\; \alpha \Big(\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{c}) - \varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \Big)$$

Use $\tilde{\varepsilon}$ in the sampler in place of the model's raw output.

Read off the special cases:

- $\alpha = 0$: ignore the prompt entirely
- $\alpha = \mathbf{1}$: exactly the plain conditional model - **no guidance**
- $\alpha > 1$: push further in the prompt's direction than the model itself would

Watch the convention. Some papers put the *conditional* prediction as the base term, which shifts what "no guidance" means. In the form above - the one `diffusers` uses - $\alpha = 1$ is neutral, and its default `guidance_scale` is 7.5.

Always check which convention a codebase means before comparing numbers.

By hand: one guidance step

One pixel. The unconditional model predicts $\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = 0.90$; told the prompt, it predicts $\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{c}) = 1.15$.

The prompt's own contribution is the difference:

$$1.15 - 0.90 = 0.25$$

Guidance scales *that*, and adds it back:

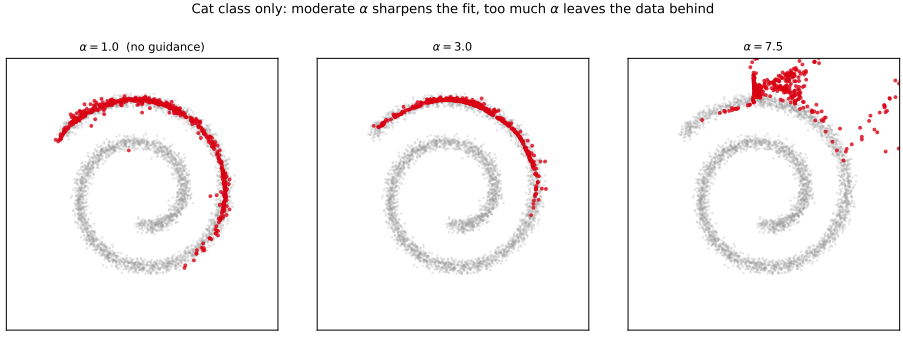
α	$\tilde{\varepsilon} = 0.90 + \alpha(0.25)$	
0	0.900	prompt ignored
1	1.150	**= the conditional model**
3	1.650	
7.5	2.775	the usual working range

Check the $\alpha = 1$ row against the formula on the last slide: the base term and the subtracted term cancel, leaving exactly $\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{c})$. **That is why $\alpha = 1$ means no guidance**, and not $\alpha = 0$.

Note how far $\alpha = 7.5$ pushes: 2.775 against a raw prediction of 1.15. We are asking the sampler to remove more than twice the noise the model actually thinks is there. It works - but you can see why it eventually stops working.

On the spiral: the cost curve

Sampling the outer class only, at three guidance strengths:

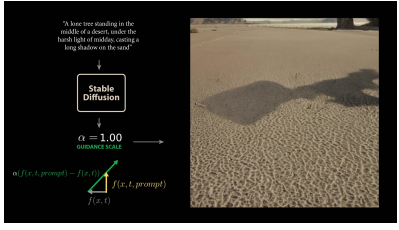


Three classes split by radius is an **easy** problem, so conditioning alone already lands on the right arm at $\alpha = 1$. The toy does not reproduce the missing tree - the Stable Diffusion image does that.

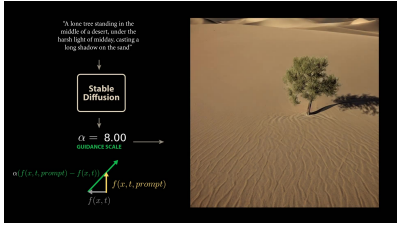
What it *does* show is overdoing it. At $\alpha = 7.5$ the samples **leave the spiral altogether** - mean radius $0.91 \rightarrow 1.10$, past the edge of the data.

And on a real model

Same prompt, same weights, same sampler. **Only α changes.**



$\alpha = 1$ - no guidance. Desert, shadow, **no tree**.



$\alpha = 8$ - the tree is there.

We changed one number. The prompt already pointed at a tree; guidance just amplified that direction.

Practically every image model you have used runs with guidance on. It is the difference between "vaguely related to your prompt" and "your prompt".

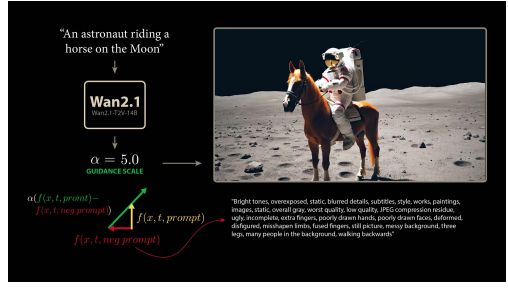
Welch Labs / 3Blue1Brown (2025)

Negative prompts: subtract what you do not want

Nothing forces the base term to be the *unconditional* prediction. Replace it with the prediction for a prompt describing what you want to **avoid**:

$$\tilde{\varepsilon} = \varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{c}_{\text{neg}}) + \alpha\big(\varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{c}) - \varepsilon_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{c}_{\text{neg}})\big)$$

Now you are steering *away* from a named region as well as toward one.



Welch Labs / 3Blue1Brown (2025)

Wan 2.1's standard negative prompt lists things like extra fingers, deformed limbs, and people walking backwards - and is written in Chinese.

The bill for all this

Guidance spends diversity to buy obedience.

High α concentrates samples in the region the prompt describes most typically. Ask for "a cat" at $\alpha = 15$ and you get the same few cats.

And it doubles the cost. Each step needs *two* forward passes - conditional and unconditional - so a guided 25-step sample is 50 network calls.

Typical values sit around 5-8. Beyond that images get oversaturated and start to look the same.

The spiral showed the far end of this: past a certain α the samples stopped resembling the data at all. On real models the same overshoot shows up as blown-out colour and hard edges rather than as points leaving a curve.

Recap

1. CLIP trains a text and an image encoder into one space, contrastively - pull true pairs together, push the rest apart.
2. The resulting geometry supports arithmetic on concepts, and zero-shot classification. But it only maps *into* the space.
3. Conditioning feeds the prompt to the denoiser - as a **sequence** of token vectors, read by **cross-attention**.
4. Conditioning alone is too weak: correct scene, missing subject.
5. Drop the label sometimes in training, and one model gives both a conditional and an unconditional prediction.
6. Amplify the difference: **classifier-free guidance**. This is why prompts work.

Next: we can now generate what we ask for.

But all of this runs in full pixel space, with two network passes per step. At 512×512 that is far beyond a laptop - and yet Stable Diffusion runs on one.

L31: how, plus video, and what all this actually costs.